

ReliefLink

Real-Time Disaster Response & Humanitarian Relief Coordination Platform

Complete Engineering Architecture, React Hooks, State Management, API Connectivity, Security Protocols & Viva Defense Manual

AUTHOR / DEVELOPER

Vivek

CORE TECH STACK

MERN + Socket.io + Leaflet GIS

FRONTEND STATE & ROUTING

Zustand + React Router v6

DOCUMENT PURPOSE

**Technical Viva & Viva-Voce
Master Guide**

Table of Contents

1. React Functional Components & Hook States (useState, useEffect)	Page 3
2. Global State Management: Why Zustand Over Redux?	Page 5
3. Frontend-to-Backend API Architecture (Axios, Interceptors, Try/Catch)	Page 7
4. Client-Side Routing: React Router DOM & Protected Route Guards	Page 9
5. Express Backend Architecture: MVC Controllers & HTTP Status Codes	Page 11
6. The Complete End-to-End Login Workflow (Step-by-Step A to Z)	Page 13
7. JSON Web Tokens (JWT) & Authentication Security Deep-Dive	Page 15
8. Critical Viva Question: "Will ReliefLink Work Without Express?"	Page 17
9. Geospatial Telemetry, 3D WebGL Earth & Matching Algorithms	Page 19
10. Relief Camp Logistics, Stock Auto-Deductions & Smart Donations	Page 21
11. Master Screen Inventory & Core System User Flows	Page 23
12. Rapid Revision: Top 25 Viva Questions & Direct Answers	Page 25

How to Use This Handbook

This document covers the **exact line-by-line codebase** of ReliefLink. Every code snippet in this manual exists directly in the project files (`frontend/src/` and `backend/`). Use this manual to explain your code with complete technical authority during external examinations and viva reviews.

Chapter 1: React Functional Components & Hook States

1.1 Why Functional Components Instead of Class Components?

In modern React (React 18.3+), all components in **ReliefLink** are authored as **Functional Components** using JavaScript functions that return JSX (JavaScript XML). We eliminated legacy Class components for the following reasons:

Aspect	Legacy Class Components	Modern Functional Components (ReliefLink)
State Syntax	Requires <code>this.state</code> and <code>this.setState()</code>	Clean <code>useState()</code> hook with explicit setters
Lifecycle Logic	Fragmented across <code>componentDidMount</code> , <code>componentDidUpdate</code> , <code>componentWillUnmount</code>	Unified cleanly within a single <code>useEffect()</code> hook with dependency control
Memory & Bundle	Heavier prototype chain; complex <code>this</code> binding	Lightweight closures; faster minification and smaller bundle size
Reusability	Requires Higher-Order Components (HOC) or Render Props	Simple Custom Hooks (e.g., <code>useAuthStore()</code> , <code>useLocation()</code>)

1.2 Deep-Dive: `useState` Hook

`useState` is the primary hook used to store and update local, component-scoped data that triggers a UI re-render whenever modified.

Code Example from ReliefLink (`frontend/src/pages/auth/LoginPage.jsx`):

```
// Declare state variables with initial default values
const [email, setEmail] = useState('');
const [password, setPassword] = useState('');
const [showPassword, setShowPassword] = useState(false);
const [responderLoading, setResponderLoading] = useState(false);

// When user types in the input box:

```

How it works internally:

- `email` is the current state snapshot.
- `setEmail` is the dispatcher function. Calling `setEmail(newValue)` schedules a state update and enqueues a re-render of the component with the new value.
- React compares virtual DOM trees (reconciliation) and updates only the modified DOM node in the browser.

1.3 Deep-Dive: `useEffect` Hook

`useEffect` allows functional components to perform side-effects, such as data fetching, subscribing to WebSocket events, or reading coordinates on page mount.

The Anatomy of `useEffect` :

```
useEffect(() => {  
  // 1. Code executed on mount or dependency change  
  
  return () => {  
    // 2. Optional cleanup function executed before re-running or on unmount  
  };  
}, [dependencies]); // 3. Dependency Array
```

Real Code from ReliefLink (`frontend/src/App.jsx`):

```
useEffect(() => {  
  if (!isAuthenticated) return;  
  
  // Connect to WebSocket channel  
  const socket = io(SOCKET_URL, { transports: ['websocket', 'polling'] });  
  
  socket.on('sos:new', (data) => {  
    addAlert(data); // update global alert state  
  });  
  
  // Cleanup function: disconnect socket when user logs out or leaves  
  return () => {  
    socket.disconnect();  
  };  
}, [isAuthenticated]); // Re-runs ONLY when auth status changes!
```

Dependency Array Rules (Viva Favorite!):

- **No dependency array:** Runs after *every single render* (risk of infinite loops).
- **Empty array `[]` :** Runs *once* when the component first mounts (like `componentDidMount`).
- **Array with values `[isAuthenticated]` :** Runs only when the specified variables change.
- **Return callback:** Executes when component unmounts to prevent memory leaks.

Chapter 2: State Management — Why Zustand Over Redux?

2.1 The Need for Global State Management

In large-scale applications like ReliefLink, multiple independent pages (e.g., `App.jsx`, `Navbar.jsx`, `SOSPage.jsx`, `NearbyRequestsPage.jsx`) all need to know: *Who is the logged-in user? What is their role? What is their JWT token?*

Without global state management, we would have to pass props down 10 levels of components (known as **Props Drilling**), which is error-prone and unmaintainable.

2.2 Why We Chose Zustand for ReliefLink

Comparison Criteria	Redux / Redux Toolkit	Zustand (ReliefLink Choice)
Library Bundle Size	~15 KB to 25 KB + dependencies	~1.5 KB (10x lighter!)
Boilerplate Code	High: Actions, Reducers, Dispatch, Slices, Thunks, Providers	Zero boilerplate: Just one <code>create()</code> function
Context Wrapper	Requires wrapping <code><Provider store={store}></code> around App	No provider required! Can be used anywhere (even outside React)
LocalStorage Sync	Requires <code>redux-persist</code> plugin and complex config	Built-in <code>persist</code> middleware works out of the box
Re-render Performance	Can trigger unnecessary tree renders without memoized selectors	Strict atomic state subscriptions

2.3 Line-by-Line Code Breakdown: `authStore.js`

Below is the exact code from `frontend/src/store/authStore.js`:

```
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

export const useAuthStore = create(
  persist(
    (set, get) => ({
      // State properties
      user: null,
```

```

    token: null,
    isAuthenticated: false,

    // Actions
    setAuth: (user, token) =>
      set({ user, token, isAuthenticated: true }),

    updateUser: (updatedUser) =>
      set({ user: { ...get().user, ...updatedUser } }),

    logout: () =>
      set({ user: null, token: null, isAuthenticated: false }),
  }),
  {
    name: 'relieflink-auth', // Key in localStorage
    partialize: (state) => ({
      user: state.user,
      token: state.token,
      isAuthenticated: state.isAuthenticated,
    }),
  }
)
);

```

Key Elements to Explain to Examiners:

1. **create()** : Initializes the store and returns a React hook (**useAuthStore**).
2. **persist()** : Middleware that automatically serializes state into browser **localStorage** under the key **relieflink-auth** .
3. **set()** : Merges new updates into the store and notifies subscribed components.
4. **get()** : Accesses the current snapshot of store values without causing a re-render.

Chapter 3: Frontend-to-Backend API Connectivity

3.1 The Communication Pipeline

The client (React) runs in the user's browser, while the server (Express) runs in Node.js. They communicate using **JSON over HTTP REST APIs** and **WebSockets**.

To handle all HTTP calls cleanly, we created a centralized Axios instance in `frontend/src/api/index.js`.

3.2 Axios Interceptors: Request & Response Guards

```
import axios from 'axios';
import { useAuthStore } from '../store/authStore';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || '/api',
  headers: { 'Content-Type': 'application/json' },
  timeout: 15000, // 15-second timeout for disaster field networks
});

// 1. REQUEST INTERCEPTOR: Automatically attaches Bearer Token
api.interceptors.request.use(
  (config) => {
    const token = useAuthStore.getState().token;
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// 2. RESPONSE INTERCEPTOR: Handles 401 Unauthorized globally
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Token expired or invalid → auto logout and redirect
      useAuthStore.getState().logout();
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

3.3 The Frontend `try ... catch` Pattern

Whenever an API call is executed in a React component, we wrap it in a strict `try ... catch ... finally` block:

```
const handleResponderLogin = async (e) => {
  e.preventDefault();
  setResponderLoading(true); // 1. Set Loading UI spinner

  try {
    // 2. Await asynchronous API network promise
    const res = await authAPI.login(email, password);

    // 3. Success handling: update global state & notify user
    setAuth(res.data.user, res.data.token);
    toast.success(`Welcome back, ${res.data.user.name}!`);
    navigate('/dashboard');
  } catch (err) {
    // 4. Error handling: extract backend validation error message
    const errorMsg = err.response?.data?.message || 'Login failed. Check credentials.';
    toast.error(errorMsg);
  } finally {
    // 5. Always executed: reset loading spinner
    setResponderLoading(false);
  }
};
```

Chapter 4: React Router DOM & Route Security Guards

4.1 Single Page Application (SPA) Routing

In traditional multi-page web applications, clicking a link requests a new HTML document from the server, causing the browser to flicker and reload. In **ReliefLink**, we use `react-router-dom` (v6) to implement **Single Page Application (SPA) Routing**.

The browser downloads the JavaScript bundle once. When navigating between `/sos` and `/volunteer`, React Router dynamically mounts and unmounts components without refreshing the page.

4.2 Core React Router Hooks & Components

- `BrowserRouter` : Wraps the entire application and synchronizes the UI with the browser's URL using the HTML5 History API (`pushState` , `replaceState`).
- `Routes` & `Route` : Defines path-to-component mappings (e.g., `<Route path="/login" element={<LoginPage />} />`).
- `useNavigate()` : A hook providing programmatic navigation (e.g., `navigate('/volunteer')` or `navigate(-1)` for back).
- `useLocation()` : A hook providing details about the current URL and state passed during navigation (e.g., `location.state?.from`).

4.3 Protected Route & Role Guard Architecture

To prevent unauthorized access (e.g., preventing a regular citizen from accessing the NGO Camp Inventory dashboard), we implemented a custom higher-order wrapper component called

`ProtectedRoute` in `frontend/src/App.jsx` :

```
function ProtectedRoute({ children, allowedRoles }) {
  const { isAuthenticated, user } = useAuthStore();
  const location = useLocation();

  // 1. Check if user is logged in
  if (!isAuthenticated) {
    // Redirect to login and remember previous page
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  // 2. Check if user possesses the required role
  if (allowedRoles && !allowedRoles.includes(user?.role)) {
    // Redirect user to their own authorized portal
    return <Navigate to={getRoleHome(user?.role)} replace />;
  }

  // 3. User is authenticated and authorized → Render Protected Page
```

```
    return children;
  }
```

Route Guard Usage in ReliefLink:

```
<Route path="/ngo/inventory" element={<ProtectedRoute allowedRoles={['ngo', 'admin']}>
  <InventoryPage /></ProtectedRoute>} />
```

If a user with role `volunteer` tries to visit `/ngo/inventory`, the guard intercepts the request and safely reroutes them to `/volunteer`.

Chapter 5: Express Backend & MVC Controllers

5.1 Model-View-Controller (MVC) Architecture

ReliefLink's backend adheres strictly to the **MVC Architecture**:

- **Model** (`backend/models/`): Defines Mongoose data schemas, field types, validation, and database indexes (e.g., `User.js` , `SosRequest.js`).
- **View** (`frontend/src/`): The React user interface that renders UI and collects user inputs.
- **Controller** (`backend/controllers/`): Contains the core business logic, database queries, and response dispatching.
- **Route** (`backend/routes/`): Maps incoming HTTP URLs and methods to specific controller functions.

5.2 Anatomy of an Express Controller Function

Every controller function accepts two fundamental parameters: `req` (Request Object) and `res` (Response Object):

- `req.body` : Contains JSON data sent via POST/PUT from the frontend.
- `req.params` : Route parameters defined with colons (e.g., `/api/sos/:id` → `req.params.id`).
- `req.query` : URL query parameters (e.g., `/api/camps?district=Ernakulam` → `req.query.district`).
- `req.user` : Injected by the `protect` middleware containing verified user data from the JWT.
- `res.status(code).json(payload)` : Dispatches the HTTP status code and JSON payload back to client.

5.3 Real Controller Code: `authController.js`

```
// @desc Login user with email/password
// @route POST /api/auth/login
const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    // 1. Validation
    if (!email || !password) {
      return res.status(400).json({ success: false, message: 'Email and password required' })
    }

    // 2. Query MongoDB with explicit password selection
    const user = await User.findOne({ email }).select('+password');

    // 3. Bcrypt Password Verification
    if (!user || !(await user.comparePassword(password))) {
```

```

    return res.status(401).json({ success: false, message: 'Invalid email or password' });
  }

  // 4. Issue JWT Token and Send Response
  const token = generateToken(user._id);
  res.status(200).json({
    success: true,
    token,
    user: { _id: user._id, name: user.name, email: user.email, role: user.role }
  });
} catch (error) {
  // 5. Prevent Node.js server crash on unhandled exceptions
  res.status(500).json({ success: false, message: error.message });
}
};

```

5.4 HTTP Status Codes Reference Table

Status Code	Meaning	When Used in ReliefLink
200 OK	Success	Successful login, fetching camps, fetching SOS records
201 Created	Resource Created	New SOS beacon created, user registered, donation recorded
400 Bad Request	Client Validation Error	Missing required phone/email, invalid OTP length
401 Unauthorized	Authentication Failure	Incorrect password, expired JWT token, missing Bearer header
403 Forbidden	Authorization Failure	Volunteer trying to approve NGO camp inventory dispatch
404 Not Found	Resource Missing	Camp ID or SOS Request ID does not exist in MongoDB
500 Server Error	Unhandled Server Bug	Database connection dropped, unexpected code exception

Chapter 6: The Complete End-to-End Login Workflow

6.1 Step-by-Step Execution Journey (A to Z)

Here is the exact lifecycle of what happens when a user logs into ReliefLink, from the moment they click the button until they see their dashboard:

1 User Input Capture (Frontend)

The user types their email and password into `LoginPage.jsx`. Each keystroke triggers `setEmail()` or `setPassword()` in React's `useState` hook.

2 Client-Side Validation & Loading State

The user submits the form. `handleResponderLogin()` runs regex checks on the email and verifies password length ≥ 6 . It triggers `setResponderLoading(true)` to display a spinner.

3 Axios HTTP POST Request Dispatch

Axios sends an asynchronous HTTP POST request to `http://localhost:5000/api/auth/login` containing `{ email, password }` in the request body.

4 Express Route & Controller Execution

The request arrives at the backend. `routes/auth.js` matches `/login` and calls `authController.login(req, res)`.

5 MongoDB Query & Bcrypt Verification

Mongoose executes `User.findOne({ email }).select('+password')`. If found, it calls `bcrypt.compare(enteredPassword, hashedPassword)`. The salt and hash are mathematically evaluated.

6 Cryptographic JWT Token Generation

If password matches, the server signs a JSON Web Token using `jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '7d' })`.

7 HTTP 200 JSON Response

The backend responds with HTTP status 200: `{ success: true, token: "eyJ...", user: { _id, name, role } }`.

8 Zustand Store & LocalStorage Persistence

The frontend receives the response. It calls `setAuth(res.data.user, res.data.token)`. Zustand's `persist` middleware automatically writes the credentials into `localStorage['relieflink-auth']`.

9 Role-Based Programmatic Navigation

The client calls `navigate('/volunteer')` or `navigate('/ngo')` based on `user.role`. A green success toast appears.

10 Subsequent Authorized API Calls

For every subsequent API call, the Axios Request Interceptor extracts the token from Zustand and injects `Authorization: Bearer <token>` into headers automatically.

Chapter 7: JSON Web Tokens (JWT) & Security Deep-Dive

7.1 What is a JSON Web Token (JWT)?

A JSON Web Token is a compact, URL-safe means of representing claims to be transferred between two parties. A JWT consists of three parts separated by dots (`.`):

```
// Format: Header.Payload.Signature
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY2ZGUxMmZh...iwiZXhwIjoxNzI2...
.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

1. Header (Algorithm & Token Type)

Specifies the cryptographic algorithm used to sign the token (e.g., `HMAC SHA256` or `RSA`) and token type (`JWT`).

2. Payload (Claims & Data)

Contains session data such as User ID (`{ id: "66de ..." }`), issuance time (`iat`), and expiration timestamp (`exp`).

3. Signature (Cryptographic Tamper Protection)

Calculated by hashing the encoded Header, encoded Payload, and the server's private `JWT_SECRET` :

```
HMACHASH256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  process.env.JWT_SECRET
)
```

If an attacker alters the user role in the payload, the signature no longer matches, and the backend instantly rejects the request!

7.2 Why Stateless JWT Over Server Sessions?

Feature	Server-Side Sessions (Express-Session)	JSON Web Tokens (ReliefLink)
Storage	Stored in Server RAM or Redis database	Stored in Client (Zero server memory overhead)
Scalability	Difficult to scale across multiple servers without centralized session store	Horizontally scalable: any server with <code>JWT_SECRET</code> can verify

DB Queries	Requires database query on every single request to validate session ID	Zero DB query needed for signature verification
------------	--	---

7.3 Authentication Middleware Code: `middleware/auth.js`

```
const protect = async (req, res, next) => {
  try {
    let token;
    if (req.headers.authorization?.startsWith('Bearer')) {
      token = req.headers.authorization.split(' ')[1];
    }
    if (!token) return res.status(401).json({ message: 'Not authorized, no token' });

    // Cryptographically verify signature
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // Attach verified user object to Express request
    req.user = await User.findById(decoded.id).select('-password');
    next(); // Proceed to controller
  } catch (error) {
    return res.status(401).json({ message: 'Token verification failed' });
  }
};
```

Chapter 8: Critical Viva Question: "Will ReliefLink Work Without Express?"

The Exact Question Asked by Examiners:

"Express illel ith work aavumo? If the Express backend server is turned off, will your project function?"

8.1 The Direct Answer:

"The Frontend User Interface will load visually in the browser, BUT none of the core functional features will work."

8.2 Detailed Technical Justification:

To impress the examiner, explain the distinct boundaries between Client-Side (React) and Server-Side (Node/Express):

Project Capability	Can React Do It Alone?	Why Express is Mandatory
Visual UI & Navigation	YES	React compiles to static HTML, CSS, and JS. The browser can render the buttons, text, maps, and 3D globe.
MongoDB Database Connection	NO	Browsers cannot open direct TCP socket connections to MongoDB Atlas. Doing so would expose database credentials publicly, a fatal security flaw.
Password Hashing (Bcrypt)	NO	Bcrypt must run on a secure server environment. If run on the client, users could intercept and tamper with hashes.
Issuing Cryptographic JWTs	NO	JWT requires signing with <code>process.env.JWT_SECRET</code> . Storing this secret key on the frontend would allow anyone to forge admin tokens.
Real-Time WebSocket Dispatch	NO	Socket.io requires a central server hub to receive a victim's SOS beacon and broadcast it to all active volunteers.

8.3 What Exactly Happens When Express is Off?

- Pressing "Login" → Axios throws `net::ERR_CONNECTION_REFUSED` (Port 5000 is unreachable). The catch block triggers `toast.error('Network Error')`.

- Pressing "**SOS Broadcast**" → Coordinates are captured by browser GPS, but the POST request fails. No beacon is saved to MongoDB.
- Opening "**Camp Finder**" → Returns an empty map because the GET request to `/api/camps` cannot fetch the shelters.

Chapter 9: Geospatial Telemetry & Matching Algorithms

9.1 MongoDB `2dsphere` Geospatial Indexing

Standard SQL databases perform slow Pythagorean calculations across latitude/longitude columns. ReliefLink uses MongoDB's native **2dsphere indexing**, which models geometry on an earth-like sphere.

Mongoose Schema Definition (`backend/models/SosRequest.js`):

```
const sosSchema = new mongoose.Schema({
  victimId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  location: {
    type: { type: String, enum: ['Point'], default: 'Point' },
    coordinates: { type: [Number], required: true } // [longitude, latitude]
  },
  urgency: { type: String, enum: ['critical', 'high', 'medium'], default: 'high' },
  status: { type: String, enum: ['pending', 'claimed', 'resolved'], default: 'pending' }
});

// CRITICAL: Create 2dsphere index for radius proximity calculations
sosSchema.index({ location: '2dsphere' });
```

9.2 The Haversine Great-Circle Distance Formula

To calculate the spherical distance between a volunteer's current GPS position (ϕ_1, λ_1) and a victim's distress pin (ϕ_2, λ_2) , we implemented the **Haversine Formula** in JavaScript:

```
function calculateDistance(lat1, lon1, lat2, lon2) {
  const R = 6371; // Earth's radius in kilometers
  const dLat = (lat2 - lat1) * (Math.PI / 180);
  const dLon = (lon2 - lon1) * (Math.PI / 180);

  const a =
    Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos(lat1 * (Math.PI / 180)) *
    Math.cos(lat2 * (Math.PI / 180)) *
    Math.sin(dLon / 2) * Math.sin(dLon / 2);

  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return R * c; // Distance in km
}
```

Why Haversine Over Google Maps Distance Matrix API?

1. **Zero Latency:** Computes locally in RAM in $O(1)$ time (<0.1 ms).
2. **Zero Cost:** No commercial API billing per query.

3. **Disaster Reality:** Road-network algorithms fail when bridges and highways are flooded. Great-circle distance gives true spatial proximity for rescue boats and aerial drones.

Chapter 10: Camp Logistics & Smart Donation Routing

10.1 Live Camp Inventory & Auto-Deductions

Each relief camp manages essential humanitarian categories in MongoDB: *Drinking Water, Food Rations, Medical Kits, Baby Food, and Blankets*.

- **Automatic Stock Deduction:** When an NGO administrator approves an aid delivery to a volunteer, the system executes an atomic decrement on MongoDB:

```
await Inventory.findOneAndUpdate(
  { campId, itemName: 'Drinking Water' },
  { $inc: { quantity: -requestedAmount } }
);
```

- **Low-Stock Threshold Alert:** When `quantity < minThreshold`, the backend flags the inventory status as `'critical'` and fires a WebSocket alert.

10.2 The Smart Donation Allocation Algorithm

Public donors typically send money and supplies to famous urban relief camps, leaving remote interior camps starving. ReliefLink's **Smart Allocation Algorithm** calculates an **Urgency Score (\$U\$)** for every camp:

$$\$U = \left(\frac{\text{Current Occupancy}}{\text{Total Capacity}} \right) \times \left(1 + \frac{\text{Ration Deficit}}{\text{Total Safety Buffer}} \right) \$$$

When a donor chooses *"General Allocation"* on the public donation page, the platform auto-routes their contribution to the camp with the highest \$U\$ score, ensuring life-saving equity across all disaster zones.

10.3 Mock Payment Gateway Simulation

In `frontend/src/components/MockPaymentGateway.jsx`, we built a client-side payment simulator that replicates actual payment gateway protocols (UPI QR, Credit/Debit card validation, simulated transaction latency, and instantaneous verified receipt generation).

Chapter 11: Master Screen Inventory & Core System Flows

11.1 Key Screen Breakdown

Screen / Route	Target User Role	Primary Capabilities
Landing Page (/)	Public / All	Interactive 3D WebGL Earth globe, live crisis summary statistics, instant emergency helpline speed-dialers.
Auth Page (/login , /register)	All Stakeholders	Phone OTP login tab for affected victims; Email/Password tab for volunteers and NGO admins; Demo role auto-fill.
Affected Dashboard (/dashboard)	Affected Citizens	1-Click Red SOS Broadcaster with auto-GPS lock, relief supply request forms, safety status timeline updater.
Global Safety Map (/map)	All / Coordinators	Interactive Esri satellite basemap with color-coded distress beacons, active shelters, and citizen condition pins.
Volunteer Terminal (/volunteer)	Volunteers	Proximity radar with 5km–50km radius slider, skill badges filter, 1-click rescue task claiming, navigation coordinates.
Camp Management (/ngo)	NGO / Admins	Camp capacity vs bed occupancy counters, warehouse stock tracking with threshold alerts, victim aid request approval.
Public Donation Portal (/donate)	Public Donors	Monetary giving via simulated UPI/Cards, physical goods drop-off pledge creator, Smart Camp auto-routing algorithm.

Chapter 12: Top 25 Viva Questions & Rapid Answers

Q1: What is a React Hook?

A: Functions introduced in React 16.8 that allow functional components to use state and lifecycle methods without writing classes.

Q2: Can we call hooks inside loops or conditions?

A: No! Hooks must always be called at the top level of a React component to maintain call order across renders.

Q3: What is the purpose of Zustand's `persist` middleware?

A: It automatically synchronizes store state with `localStorage`, so the user stays logged in after page refresh.

Q4: Why does GeoJSON require `[longitude, latitude]` order?

A: Because in Cartesian geometry, X comes before Y. Longitude is horizontal (X), Latitude is vertical (Y).

Q5: What is CORS?

A: Cross-Origin Resource Sharing. A browser security mechanism that blocks requests from different origins (e.g., React on port 5173 calling Express on port 5000) unless allowed via the `cors` middleware.

Q6: How are passwords encrypted in ReliefLink?

A: They are not encrypted (reversible); they are one-way **hashed and salted** using `bcryptjs` with 10 salt rounds.

Q7: What is the difference between `res.send()` and `res.json()` ?

A: `res.json()` explicitly sets the `Content-Type: application/json` header and formats JavaScript objects into JSON strings.

Q8: What is the Haversine formula used for?

A: To calculate the shortest great-circle distance between two GPS coordinate points on a spherical Earth.

Q9: How do you prevent race conditions when two volunteers accept the same SOS task?

A: We use MongoDB atomic updates: `findOneAndUpdate({ _id: id, status: 'pending' }, { status: 'claimed' })`.

Q10: What does the `next()` function do in Express middleware?

A: It passes execution control to the next middleware or controller in the HTTP request processing pipeline.